Assignment 5: Optimising Attention on a CPU

CS6886: Systems for Deep Learning, IIT Madras

Q1. Machine and roofline parameters

Machine details are in `machine_spec.txt`. The CPU is an Intel Core i5-1135G7 with 4 physical cores, 8 logical processors, AVX2 (8 FP32 lanes per 256-bit vector) and two FMA units per core. One FMA counts as two FLOPs.

Quantity	Calculation	Result
One-core FP32 peak	4.20 GHz × 8 lanes × 2 FMA units × 2 FLOP	134.4 GFLOP/s
All-core FP32 peak	134.4 × 4 physical cores	537.6 GFLOP/s
Memory bandwidth	3200 MT/s × 8 B × 2 channels	51.2 GB/s
One-core ridge point	134.4/51.2	2.63 FLOP/B
All-core ridge point	537.6/51.2	10.5 FLOP/B

These are theoretical limits. The benchmark is single-process code on Windows, and the measured kernel also pays for reductions, `expf`, loop overhead and memory effects.

Q2. Naive baseline

The driver does one warm-up run and then five timed runs, and reports the median. FLOPs are counted as $4N^2d$. Percentage of peak is GFLOP/s divided by 134.4.

N	d	Time (ms)	GFLOP/s	% of one-core peak
1024	64	139.780	1.92	1.43
2048	64	544.247	1.97	1.47
4096	64	2151.575	2.00	1.49
4096	128	6816.338	1.26	0.94

Scaling. The main work is $O(N^2d)$. Doubling N from 1024 to 2048 costs $544.247/139.780 = 3.89\times$, and from 2048 to 4096 costs $2151.575/544.247 = 3.95\times$, both close to the ideal $4\times$. Doubling d from 64 to 128 at N=4096 costs $6816.338/2151.575 = 3.17\times$, above the ideal $2\times$. This is consistent with cache pressure from a larger working set.

Where the time goes. The naive code materialises the $N \times N$ score matrix. It computes $QK^\top$, a row-wise stable softmax, and then SV. The matrix is written, scanned several times by softmax, and read again by SV. The byte model in Q5 counts only one write and one aggregate read, so it understates the real traffic.

Q3. Single-threaded optimisations

Speedup is `naive_time / variant_time`; values below 1 mean the variant is slower than naive.

Input	Variant	Time (ms)	GFLOP/s	Speedup
N=1024, d=64	naive	139.780	1.92	1.000
	loops	129.158	2.08	1.082
	tiled	165.533	1.62	0.844
	fused	135.312	1.98	1.033
	best	103.515	2.59	1.350
N=2048, d=64	naive	544.247	1.97	1.000
	loops	507.108	2.12	1.073
	tiled	667.367	1.61	0.815
	fused	546.136	1.97	0.997
	best	376.321	2.85	1.446
N=4096, d=64	naive	2151.575	2.00	1.000
	loops	2099.369	2.05	1.025
	tiled	2519.429	1.70	0.854
	fused	2181.522	1.97	0.986
	best	1621.206	2.65	1.327
N=4096, d=128	naive	6816.338	1.26	1.000
	loops	9818.777	0.87	0.694
	tiled	7544.418	1.14	0.904
	fused	6927.238	1.24	0.984
	best	5848.476	1.47	1.165

Loops. Makes the contiguous key-row index the innermost loop of $QK^\top$, which exposes a vectorisable loop. This gives a small gain at d=64. At d=128 the changed accumulation order and cache behaviour make it slower in this run.

Tiled. Uses 32-element tiles for both products. Tiling can improve reuse, but boundary loops and extra control overhead outweigh the benefit at these sizes.

Fused. Processes one query row at a time and never allocates the score matrix. This removes allocation and score-matrix traffic, but the row-wise dot products, `expf` and value reductions remain.

Best. The fused design with a reusable row and OpenMP SIMD reduction directives (enabled with `-fopenmp-simd`, no parallel execution). This makes both dot-product reductions vectorisable, and it is the fastest single-threaded variant on all four inputs. The remaining limits are the irregular row-wise access to K and V, the serial softmax work and `expf`.

All variants passed the output tolerance check. Full data and validation status are in `results.csv`.

Q4. OpenMP

The OpenMP version parallelises independent query rows, and each thread has a private temporary weight row. Results at N=4096, d=64:

Threads	Time (ms)	Speedup vs 1 thread	Parallel efficiency
1	1615.961	1.000×	100.0%
2	845.740	1.911×	95.5%
4	665.662	2.428×	60.7%
8	452.059	3.575×	44.7%

Efficiency is speedup divided by thread count. As a sanity check, `best` and `omp` with one thread were run back to back: 1612.727 ms and 1615.961 ms. The small gap is benchmark noise plus OpenMP-region overhead, so the two use the same algorithm.

Scaling is well below ideal. Efficiency drops sharply from 2 to 4 threads, and 8 threads on 4 physical cores still gives the best time (3.58×). This suggests the row loop is limited by `expf` and reduction latency rather than by DRAM, which agrees with the roofline placing the fused algorithm on the compute side. Shared cores, cache contention and the limited throughput of the softmax work all cap the speedup.

Q5. Roofline

Every variant is assigned $4N^2d$ FLOPs: two FLOPs per multiply-add in $QK^\top$ and two in SV. Arithmetic intensity (AI) is FLOPs divided by an estimated byte count, in bytes of FP32 (4 B per element).

For variants that store the score matrix, bytes $= 4(4Nd + 2N^2)$. The $4Nd$ term covers Q, K, V, O; the $2N^2$ term approximates one score-matrix write plus one aggregate read. It is not a literal count of every softmax load and store, so real naive traffic is higher. For fused variants, bytes $= 4(4Nd)$, since only a length-N temporary row is kept.

Variant	AI formula	AI at N=4096, d=64	Main limits
naive	$\frac{4N^2d}{4(4Nd+2N^2)}$	31.03 FLOP/B	Score-matrix traffic, repeated softmax scans, `expf`, scalar reductions.
loops	same as naive	31.03 FLOP/B	Better $QK^\top$ vectorisation, but same score traffic and SV access pattern.
tiled	same formula; reuse changes real traffic	31.03 FLOP/B	Tile overhead and softmax dominate; score matrix is much larger than cache.
fused	$\frac{4N^2d}{4(4Nd)} = \frac{N}{4}$	1024 FLOP/B	Dot-product and reduction efficiency, `expf`; no large score allocation.
best	same as fused	1024 FLOP/B	Same softmax and memory-access limits, with better vectorisation.
omp	same as fused, per thread	1024 FLOP/B	Physical-core saturation, thread and cache contention, `expf`, reductions.

At N=4096, d=64 the score matrix is $4096 \times 4096 \times 4$ B = 64 MiB, far larger than the 8 MiB shared L3. Both 31.03 and 1024 FLOP/B lie right of the one-core ridge point (2.63 FLOP/B), so the idealised roofline labels them compute-bound. That does not mean the materialised variants have no memory problem: their real cache and DRAM traffic exceeds the compulsory model. Measured performance is also far below the 134.4 GFLOP/s peak. For example, the single-thread best result at N=4096, d=64 is 2.65 GFLOP/s, only 1.97% of peak. The gap comes from scalar and reduction dependencies, non-ideal vectorisation, `expf`, cache misses, loop overhead and the lack of a tuned BLAS microkernel.